

DP Computer Science: B 2.3.2 Selection Structures

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Explain the purpose and use of selection structures (**if**, **else**, **else if/elif**)
 - Apply **Boolean and relational operators** to control program flow
 - Differentiate between **if-else** and **multiple if** statements
 - Implement selection structures within **loops** and **nested loops**
-

Key Vocabulary

Term	Definition
Selection structure	A programming construct that allows a program to make decisions based on conditions
if statement	Executes a block of code only if a condition is true
else statement	Executes a block of code when the if condition is false
elif/else if	Used to check multiple conditions in sequence
Boolean operator	Used to combine conditions: <code>&&</code> (AND), <code> </code> (OR), <code>!</code> (NOT)
Relational operator	Compares values: <code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>
Nested if	An if statement inside another if statement
Short-circuit evaluation	Stops evaluating once the outcome is determined

IB ATL Skills

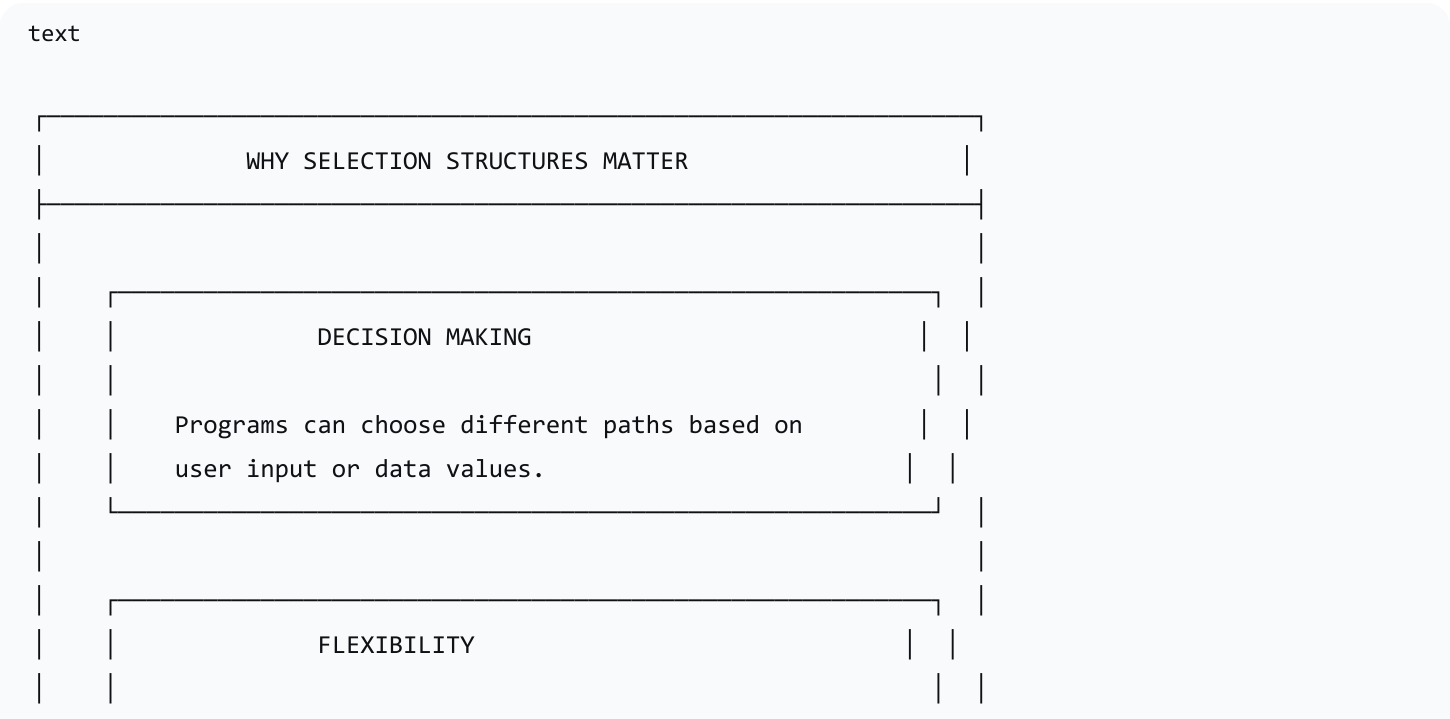
Skill	Description
Thinking	Critical thinking (analysing and evaluating issues); creative thinking (generating novel ideas)
Communication	Exchanging thoughts and information effectively through interaction
Self-Management	Managing time and tasks effectively
Research	Finding, interpreting, judging, and creating information

2. Introduction to Selection Structures

What are Selection Structures?

Selection structures allow programs to make decisions and execute different code blocks based on conditions.

Why Use Selection Structures?



Code can adapt to different situations.

CONTROL FLOW

The flow of execution can be directed to different parts of the program.

3. Basic Selection Structures

The if Statement

The `if` statement executes a block of code only if a condition is true.

python

```
# Python
age = 18
if age >= 18:
    print("You are eligible to vote.")
```

java

```
// Java
int age = 18;
if (age >= 18) {
    System.out.println("You are eligible to vote.");
}
```

The if-else Statement

The `if-else` statement provides an alternative block when the condition is false.

python

```
# Python
score = 75
if score >= 60:
    print("Pass")
else:
    print("Fail")
```

java

```
// Java
int score = 75;
if (score >= 60) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

The if-elif-else Statement

`elif` (Python) or `else if` (Java) allows checking multiple conditions in sequence.

python

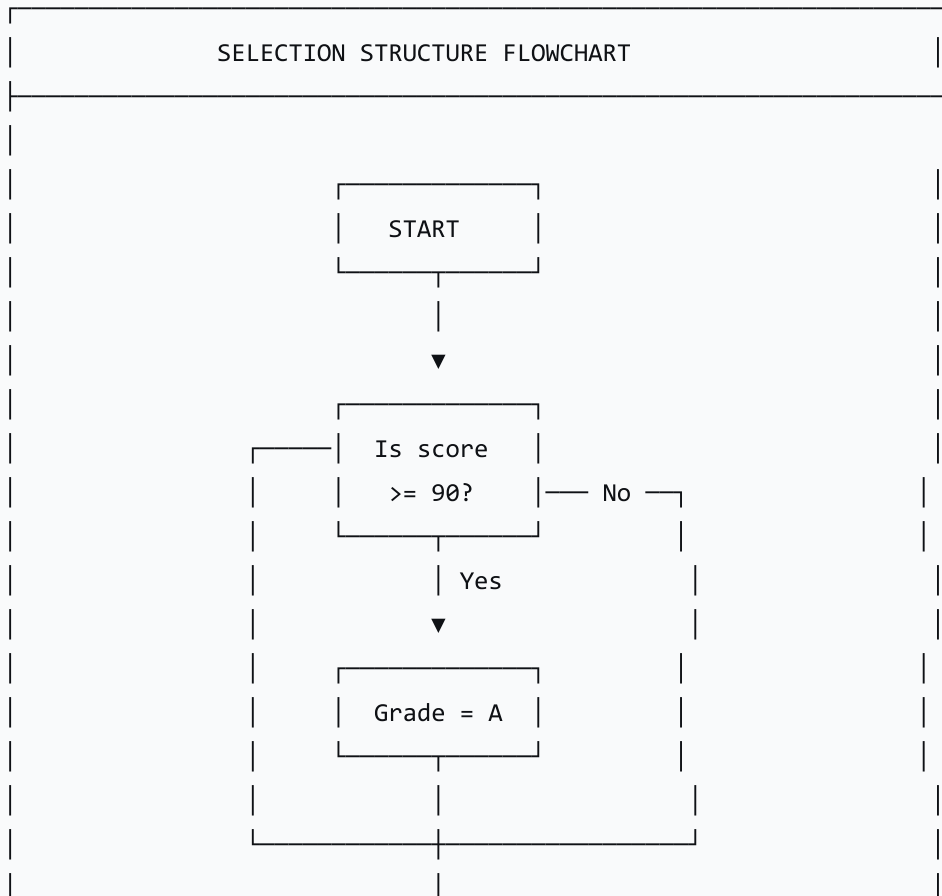
```
# Python
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
print(f"Grade: {grade}")
```

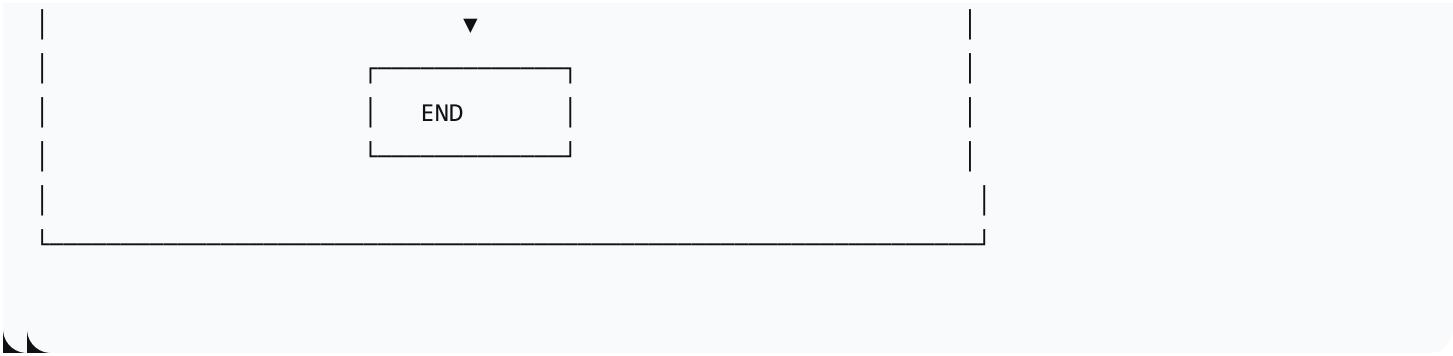
java

```
// Java
int score = 85;
char grade;
if (score >= 90) {
    grade = 'A';
} else if (score >= 80) {
    grade = 'B';
} else if (score >= 70) {
    grade = 'C';
} else if (score >= 60) {
    grade = 'D';
} else {
    grade = 'F';
}
System.out.println("Grade: " + grade);
```

Visual Representation

text





4. Boolean and Relational Operators

Relational Operators

Operator	Meaning	Example
<code>==</code>	Equal to	<code>x == 5</code>
<code>!=</code>	Not equal to	<code>x != 5</code>
<code><</code>	Less than	<code>x < 5</code>
<code>></code>	Greater than	<code>x > 5</code>
<code><=</code>	Less than or equal	<code>x <= 5</code>
<code>>=</code>	Greater than or equal	<code>x >= 5</code>

Boolean Operators

Operator	Meaning	Example
<code>and</code> (Python) / <code>&&</code> (Java)	AND (both conditions true)	<code>age >= 18 and age <= 65</code>
<code>or</code> (Python) / <code> </code> (Java)	OR (at least one condition true)	<code>age < 18 or age > 65</code>
<code>not</code> (Python) / <code>!</code> (Java)	NOT (negates a condition)	<code>not (age >= 18)</code>

Examples

python

```
# Python
age = 25
is_student = True

# AND
if age >= 18 and age <= 65:
    print("Working age")

# OR
if age < 18 or age > 65:
    print("Not working age")

# NOT
if not is_student:
    print("Not a student")
```

java

```
// Java
int age = 25;
boolean isStudent = true;

// AND
if (age >= 18 && age <= 65) {
    System.out.println("Working age");
}

// OR
if (age < 18 || age > 65) {
    System.out.println("Not working age");
}

// NOT
if (!isStudent) {
    System.out.println("Not a student");
}
```

5. Nested Selection Structures

What are Nested if Statements?

Nested if statements are if statements inside other if statements. They allow for more complex decision-making.

Nested if Example

python

```
# Python
age = 20
is_student = True

if age >= 18:
    print("Adult")
    if is_student:
        print("Adult student")
    else:
        print("Adult non-student")
else:
    print("Minor")
```

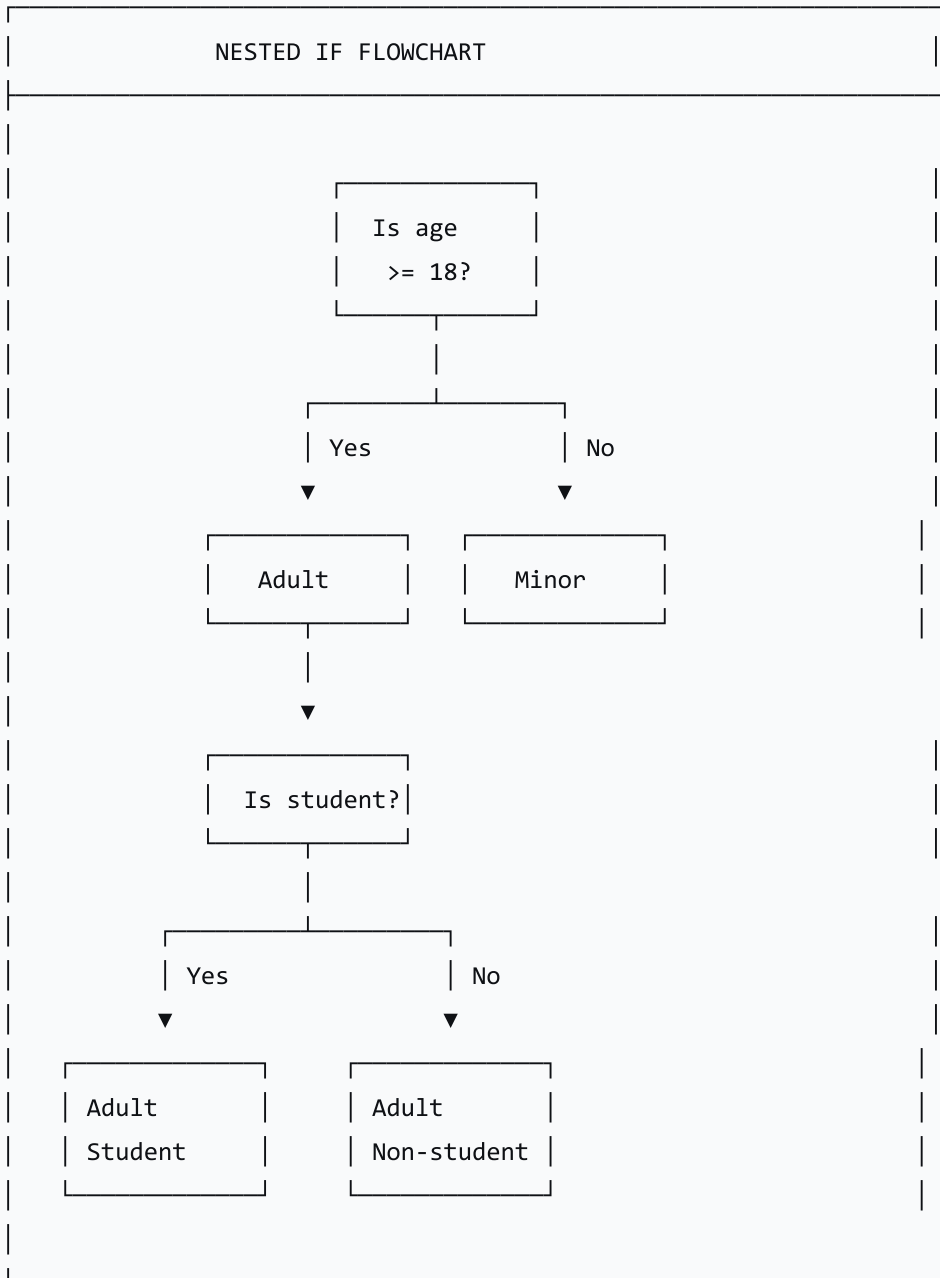
java

```
// Java
int age = 20;
boolean isStudent = true;

if (age >= 18) {
    System.out.println("Adult");
    if (isStudent) {
        System.out.println("Adult student");
    } else {
        System.out.println("Adult non-student");
    }
} else {
    System.out.println("Minor");
}
```


Nested if Flowchart

text



6. Selection Structures in Loops

Using if Inside Loops

Selection structures can be used inside loops to make decisions during iteration.

python

```
# Python: Find even numbers
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

for num in numbers:
    if num % 2 == 0:
        print(f"{num} is even")
    else:
        print(f"{num} is odd")
```

java

```
// Java: Find even numbers
int[] numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

for (int num : numbers) {
    if (num % 2 == 0) {
        System.out.println(num + " is even");
    } else {
        System.out.println(num + " is odd");
    }
}
```

Loops Inside Selection Structures

python

```
# Python: Print numbers only if a condition is met
grade = 'A'
if grade == 'A':
    for i in range(5):
        print(f"Excellent! Score: {i + 1}")
```

java

```
// Java: Print numbers only if a condition is met
char grade = 'A';
if (grade == 'A') {
    for (int i = 0; i < 5; i++) {
```

```
        System.out.println("Excellent! Score: " + (i + 1));
    }
}
```

7. if-else vs. Multiple if Statements

Difference Between if-else and Multiple if

Feature	if-else	Multiple if
Execution	Only one block executes	Multiple blocks may execute
Efficiency	More efficient (stops at first match)	Less efficient (checks all conditions)
Use Case	Mutually exclusive conditions	Independent conditions

if-else (Mutually Exclusive)

```
python

# Python: Only one block executes
x = 5
if x > 0:
    print("Positive")
else:
    print("Negative or zero")
# Output: Positive (only one block)
```

Multiple if (Independent)

```
python

# Python: Multiple blocks may execute
x = 5
if x > 0:
```

```
print("Positive")
if x < 10:
    print("Less than 10")
if x % 2 == 0:
    print("Even")
# Output: Positive, Less than 10 (Even is not printed)
```

8. Worksheet Activities

Activity 1: Grade Calculator

python

```
# Write a program that:
# 1. Asks for a student's score
# 2. Prints the grade based on:
#    - A: 90-100
#    - B: 80-89
#    - C: 70-79
#    - D: 60-69
#    - F: 0-59
```

Activity 2: Even or Odd

python

```
# Write a program that:
# 1. Asks for a number
# 2. Prints "Even" if it's even, "Odd" if it's odd
```

Activity 3: Leap Year Checker

python

```
# Write a program that:
# 1. Asks for a year
# 2. Prints "Leap year" if it's a Leap year
# 3. A year is a Leap year if:
#     - Divisible by 4
#     - Not divisible by 100 unless also divisible by 400
```

Activity 4: Nested Selection

python

```
# Write a program that:
# 1. Asks for age and whether the person is a student
# 2. Prints:
#     - "Adult student" if age >= 18 and student
#     - "Adult non-student" if age >= 18 and not student
#     - "Minor" if age < 18
```

9. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introduction	10 min	What are selection structures? if statement
else and elif	15 min	if-else, elif; Activity 1
Boolean/Relational Operators	15 min	Boolean and relational operators; Activity 2
Advanced Selection	10 min	Nested if; selection within loops
Conclusion	10 min	Key takeaways; further exploration

10. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate understanding of selection structures
Code Review	Students review each other's code

11. Differentiation

Level	Strategy
Support	Provide scaffolded examples; use visual aids
Challenge	Explore complex nested conditions; switch statements

12. Resources

- **Presentation** (Slide Deck)
 - **eBook** (B2.3.2 with examples)
 - **Worksheets:** Selection structures exercises
 - **Worksheet Answers**
-

13. Summary: Key Takeaways

Concept	Key Point
if	Execute code if condition is true

Concept	Key Point
else	Execute code if condition is false
elif	Check multiple conditions
Relational Operators	Compare values (==, !=, <, >, etc.)
Boolean Operators	Combine conditions (and, or, not)
Nested if	if statements inside if statements

text

KEY REMINDERS

- ✓ Selection structures allow programs to make decisions
- ✓ if = single condition
- ✓ if-else = two paths
- ✓ elif = multiple conditions
- ✓ Relational operators compare values
- ✓ Boolean operators combine conditions
- ✓ Nested if = if inside if

"Selection structures give programs the power to make decisions—different conditions lead to different paths."